Does a Sequence of Frames Tell You More Than One Frame?

Ego-motion classification on KITTI odometry with frozen DINOv2 features, small temporal heads and a zero-shot VLM

Tejas Sanjay Gupta · Technical report, Temporal & Multimodal Visual Intelligence assessment

Abstract

I ask whether a model can get information from a short sequence of frames that is not in any single frame. The test bed is KITTI odometry. Windows of eight frames are labelled straight, left or right from the ground-truth poses, each frame is embedded with a frozen DINOv2 encoder, and a single-frame classifier is compared with small GRU and Transformer heads. The temporal models raise test macro-F1 from about 0.42 to about 0.58, and this survives a capacity-matched control and a block bootstrap. It is not, however, evidence that the models follow the order of events: a Transformer that cannot see positions does just as well, shuffling the frames barely changes the ordered models, and removing single frames shows that they mostly compare the first frame with the rest. What they extract is how much the scene has changed. The gain does not carry over to the highway sequence, and a zero-shot 2B vision-language model stays at chance on the same windows.

1. Dataset and problem

I use the grayscale left camera (`image_0`) of the KITTI odometry benchmark, recorded at 10 Hz, with the ground-truth poses of sequences 00 to 10. The loading script checks that frame ids are contiguous, that the number of frames, poses and timestamps agree, that timestamps are strictly increasing, and that the image size is constant; it also writes per-sequence frame counts, dt statistics, class counts and displacement and yaw histograms to `results/` (`scripts/01_prepare_data.py`).

A sample is a window of 8 frames taken every second frame, so consecutive frames are 0.2 s apart and the window spans 1.4 s. The label comes from the relative pose between the first and the last frame of the window: left or right if the absolute yaw change exceeds 9 degrees, straight if it is below 3 degrees, in both cases with at least 1.5 m travelled. Windows in between (yaw of 3 to 9 degrees, slow creeping, standstill) are dropped. I added this dead zone after a first run in which many labels were decided by a fraction of a degree. The splits are by sequence, so overlapping windows never cross from training to test, and the test set contains the highway sequence 01 as a deliberate change of environment.

Split	Sequences	Windows	Straight	Left	Right
Train	00, 02, 03, 05, 08	6709	4731	915	1063
Validation	06, 09	1054	715	138	201
Test	07, 10, 01, 04	1431	1067	142	222

Table 1. Windows per split and class after the dead zone.

What changes along a sequence. While driving straight the scene expands slowly and consecutive embeddings stay close; the mean cosine similarity between consecutive frames is 0.952 for straight, 0.949 for left and 0.952 for right, so raw similarity does not separate the classes. In the turns I looked at, the lateral shift and the change in road geometry appear mostly in the later frames, and the first frames look like straight driving. The evidence for the label therefore arrives late in the window. *Where one frame is ambiguous.* In a preliminary four-class run, windows of a stationary car and of a slowly moving car had almost identical last frames (cosine 0.985 to 0.999), so standing still cannot be told from creeping in a single frame. KITTI odometry has only a few dozen stationary windows, so I dropped that class and kept the three-class task. *Where time is not needed.* Sharp turns of more than 25 degrees show their curvature in one frame, and both approaches get them right (398 of the 1431 test windows are classified correctly by both, with high confidence from the single frame).

2. Approach

Representation. Frames are passed through a frozen DINOv2 ViT-S/14 (`facebook/dinov2-small`). The grayscale image is repeated to three channels and resized to 224 by 742 pixels, which keeps the aspect ratio without cropping. A frame is represented by its class token concatenated with the mean patch token (768 values), cached once, and standardised with training-set statistics only. A window is then the sequence $E_1 \dots E_8$.

Models. All heads are small and are trained on the cached embeddings. **single** is an MLP that sees one frame (during training a random frame of the window; I report it on the last frame and averaged over the frame it is shown). **single_wide** is the same model made as large as the Transformer, as a control for capacity. **meanpool** is an MLP on the average of all frames, so it has all the information but cannot compare frames. **gru** is a one-layer GRU. **transformer** has two layers, four heads, width 128, a class token and learned position embeddings; **transformer_nopos** is identical

without positions, so it sees the frames as a set. **gru_delta** and **transformer_delta** receive each embedding together with its difference to the previous frame. Parameter counts are in Table 2.

Training and evaluation. Class-weighted cross-entropy (about 70 percent of the windows are straight), AdamW with learning rate 1e-3 and weight decay 1e-2, cosine schedule, at most 40 epochs, early stopping on validation macro-F1, three seeds. Macro-F1 is the main metric because accuracy rewards predicting the majority class. Since windows overlap, the standard deviation over seeds understates the uncertainty, so I also give 95 percent intervals from a block bootstrap (5 s blocks, 2000 resamples) of the three-seed ensemble, and paired differences with respect to the single-frame model.

3. Experiments and results

3.1 Single frame against temporal models

Model	Params	Accuracy	Macro-F1 (seeds)	Macro-F1 ensemble [95% CI]	Difference to single [95% CI]
single (last frame)	197,892	0.557 ± 0.039	0.418 ± 0.010	0.409 [0.324, 0.492]	n/a
single_wide	364,860	0.532 ± 0.062	0.407 ± 0.026	0.411 [0.321, 0.497]	+0.001 [−0.033, +0.035]
meanpool	197,892	0.565 ± 0.022	0.447 ± 0.021	0.444 [0.345, 0.538]	+0.035 [−0.006, +0.078]
gru	198,020	0.623 ± 0.017	0.581 ± 0.019	0.585 [0.480, 0.678]	+0.173 [+0.095, +0.245]
transformer	365,316	0.624 ± 0.021	0.578 ± 0.040	0.617 [0.517, 0.703]	+0.205 [+0.140, +0.272]
transformer_nopos	365,316	0.625 ± 0.011	0.584 ± 0.017	0.612 [0.509, 0.696]	+0.199 [+0.132, +0.267]
gru_delta	296,324	0.657 ± 0.006	0.577 ± 0.021	0.579 [0.454, 0.690]	+0.168 [+0.096, +0.237]
transformer_delta	463,620	0.634 ± 0.018	0.571 ± 0.027	0.587 [0.483, 0.674]	+0.175 [+0.098, +0.251]

Table 2. Test results (1431 windows). Mean ± standard deviation over three seeds, and bootstrap intervals for the seed ensemble. The single-frame row uses the last frame; averaged over the frame it is shown the single model reaches 0.444 (single_wide 0.436), so the temporal advantage is closer to +0.13 than to +0.17.

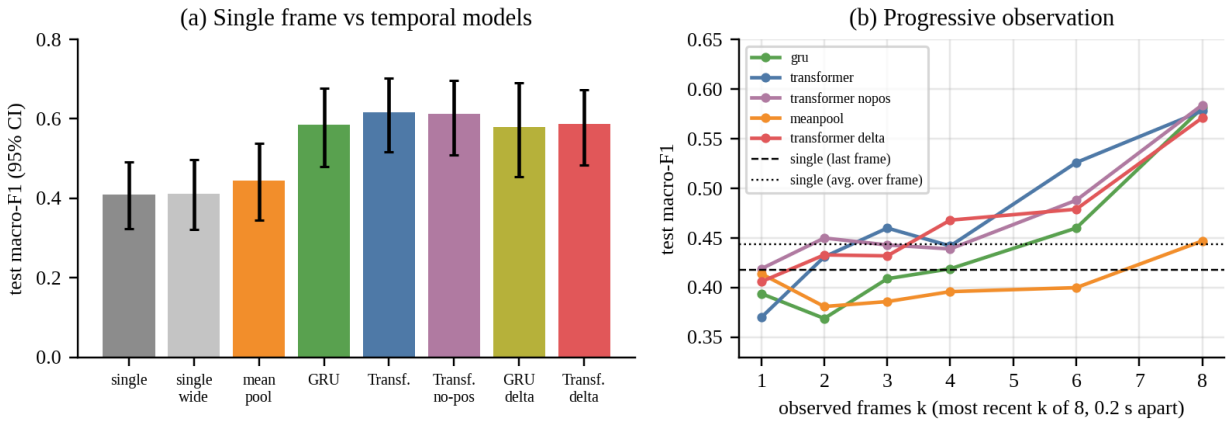


Figure 1. (a) Macro-F1 with 95 percent block-bootstrap intervals. (b) Progressive observation: one model per k, trained and tested on the most recent k of the 8 frames.

Every model that can relate frames to each other, recurrent or attention-based, improves macro-F1 by 0.17 to 0.21, and all of these intervals exclude zero. Mean pooling, which has the same frames but cannot compare them, gains 0.035 and the interval includes zero. The capacity-matched single-frame model gains nothing. So the improvement is not simply a bigger network, and it is not simply more frames. In the progressive experiment (Figure 1b) performance rises slowly up to four frames and then jumps between six and eight frames (0.46 to 0.53 at k = 6, 0.57 to 0.58 at k = 8). This is expected because the label is defined over the whole 1.4 s span; only k = 8 sees all of it. With k = 1 the temporal models fall back to the single-frame level, which is a useful consistency check.

3.2 Do the models use the order of the frames?

Model	Correct order	Shuffled	Reversed	Left/right swap rate, correct → reversed
single (last frame)	0.418	0.447	0.462	0.151 → 0.172
meanpool	0.447	0.447	0.447	0.109 → 0.109
gru	0.581	0.555	0.536	0.095 → 0.126
transformer	0.578	0.569	0.563	0.136 → 0.134
transformer_nopos	0.584	0.584	0.584	0.099 → 0.099
gru_delta	0.577	0.493	0.449	0.059 → 0.169
transformer_delta	0.571	0.523	0.482	0.102 → 0.158

Table 3. Test macro-F1 for windows fed in the correct order, randomly shuffled, or reversed in time (models trained on correct order). The swap rate is the share of left and right windows predicted as the opposite turn. If turn direction were read from temporal order, reversal should push it towards 1.

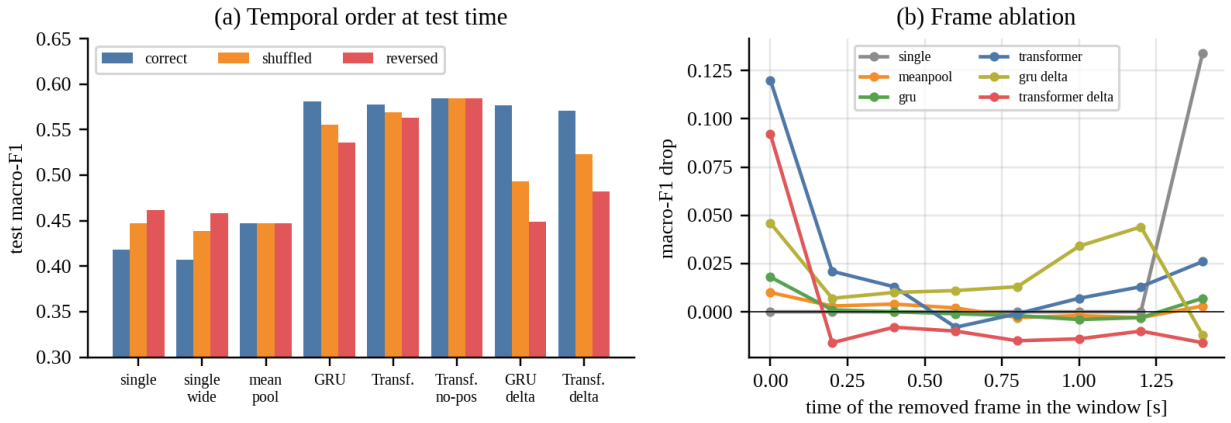


Figure 2. (a) Effect of shuffling and reversing the window at test time. (b) Frame ablation: loss in macro-F1 when one frame is replaced by the training mean.

With plain embeddings the Transformer hardly reacts to order (0.578 correct, 0.569 shuffled, 0.563 reversed), and the variant without positions matches it. The GRU shows a small effect (0.045 lower when reversed, about two seed standard deviations). The delta models, which get the frame-to-frame change explicitly, are clearly order-sensitive (0.577 to 0.449 for the GRU, 0.571 to 0.482 for the Transformer) and their left/right swap rate rises, but only from about 0.06 to 0.17, far from the value near 1 that full use of direction would give. The ablation in Figure 2b shows where the information sits: removing the first frame costs the Transformer 0.120 macro-F1 (delta variant 0.092), while no other frame costs more than 0.026; the GRU depends less on any single frame. The last-layer attention agrees (Figure 3b): the class token puts 0.168 of its weight on the first frame against 0.098 to 0.119 elsewhere (uniform would be 0.125), and the pattern hardly changes when the frames are shuffled (0.155), so it follows position, not content. My reading is that these models behave like first-versus-rest comparators. They estimate how much the view has changed since the start of the window, which is what the pose-based label measures.

3.3 Corruption, environments and failure cases

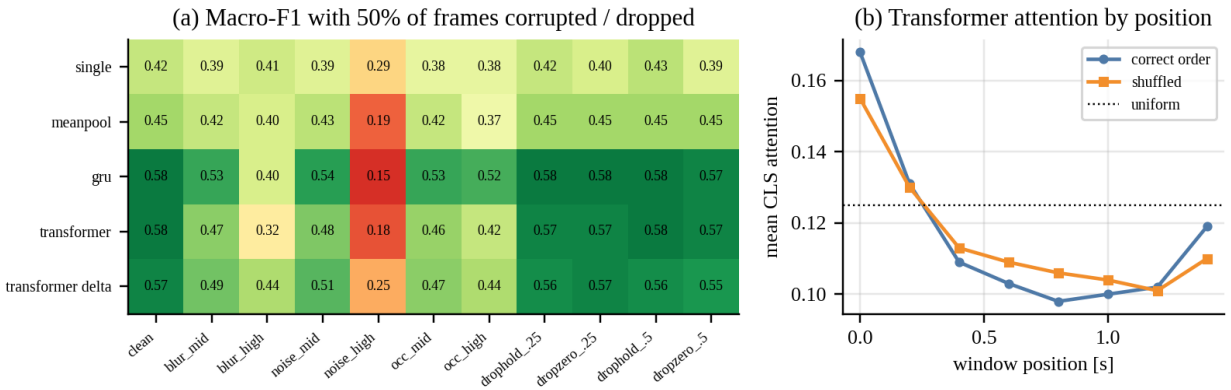


Figure 3. (a) Macro-F1 when a random half of the frames of the test sequences is blurred (sigma 3 or 8 px), corrupted with noise (std 0.10 or 0.30), partly occluded (25 or 50 percent of the image) or dropped (hold: repeat the previous embedding; zero: replace by the training mean). Models were trained on clean data. (b) Mean class-token attention at each window position.

At moderate corruption the temporal models stay above the single frame (blur_mid: Transformer 0.47, GRU 0.53, single 0.39), but they lose more in absolute terms (0.11 for the Transformer against 0.03). Under the strongest noise they fall below the single frame (0.15 to 0.25 against 0.29), presumably because corrupting only some frames creates frame-to-frame differences that a comparison-based model reads as motion. The delta Transformer is the most robust temporal model to blur (0.44 at sigma 8). Dropped frames have almost no effect, but this is too optimistic: my implementation never drops the first frame, and the ablation shows that this is the one that matters.

Excluding the highway sequence the temporal models lead clearly (macro-F1: single 0.569, single_wide 0.534, meanpool 0.626, GRU 0.689, Transformer 0.721, Transformer-delta 0.695). On sequence 01 none of the models works (0.20 to 0.25 for all of them). Its mean embedding has a cosine of about 0.4 with the other sequences, so this is a shift in the representation that temporal modelling does not repair. Comparing the ensemble Transformer with the single frame on the 1431 test windows, the temporal model fixes 218 windows, breaks 87 and both are wrong on 416. Recall for left turns goes from 0.42 to 0.76 and for right turns from 0.28 to 0.72, while straight stays at 0.63 against 0.62. Example strips for each group are written to `results/figures/cases_*.png`.

3.4 Vision-language model

I ran Qwen2-VL-2B zero-shot on 45 balanced test windows (15 per class) and asked for a single word (straight, left or right) given the last frame, the eight frames in order with frame labels, or the same eight frames shuffled. Accuracy was 0.42 for the single frame (macro-F1 0.32), 0.33 for the ordered sequence (macro-F1 0.17) and 0.36 for the shuffled sequence. With the sequence prompt the model answered straight for all 45 windows, which is exactly the chance level of 1/3. On the same windows the embedding models reach 0.38 (single frame) and 0.64 (Transformer). With 45 windows the uncertainty is around ±15 points, so I read the result as: this model has no usable ego-motion signal. The VLM does not add value for metric motion or turn direction, which depends on small geometric changes between frames. Where I expect it to help is on semantics: describing what changes and which objects appear, comparing an early and a late frame in words, and flagging windows that look ambiguous, as a layer on top of a calibrated motion estimate. The free-text outputs are saved in `results/vlm/vlm_descriptions.md`; I did not score them quantitatively.

4. Research analysis

A. When did temporal information help? On turns in urban sequences. Left recall rose from 0.42 to 0.76 and right recall from 0.28 to 0.72, and macro-F1 rose by 0.17 to 0.21 (about 0.13 against the position-averaged single frame), but only when the window covered the whole motion span (k = 8). Parameter count does not explain it, because single_wide gained +0.001.

B. When did it not help? On the straight class (recall 0.63 against 0.62), on the highway sequence (all models at or below 0.25 macro-F1), under strong noise (worse than the single frame), for the intermediate frames (removing any of them costs at most 0.03), and as order information when plain embeddings are used (shuffled and reversed windows are about as good as correct ones).

C. How can one tell whether a temporal model uses temporal relationships and not just more visual information? I compare it with models that receive the same frames without order: mean pooling, which cannot compare frames (+0.035, not significant), and the Transformer without positions, which can compare frames but not order (same score as the ordered model). Then I intervene on order (shuffle, reverse, left/right swap rate) and on content (frame ablation, progressive k). The outcome here is that the gain comes from comparing frames, mostly the first with the rest. Real use of order appears only with explicit difference features, and only partly.

D. What result would show that the temporal approach is not useful? A temporal model that fails to beat a capacity-matched, position-averaged single-frame model with a paired interval that excludes zero, or that keeps its score when all frames but one are removed or replaced by copies of one frame, or that does not improve as k grows. The first test fails here, since the temporal models win, so on this task the temporal approach is useful, although only as a shallow endpoint comparison.

E. What would give stronger evidence? A task whose label depends on order and not on endpoints (for example left-to-right against right-to-left motion, or a label defined on the last 0.4 s inside a longer window); a two-frame baseline using only the first and last frame, which tests the endpoint explanation directly; more turns, more environments, leave-one-sequence-out validation and more than three seeds; motion-aware features (optical flow, spatially resolved patch features) instead of a global token; replacing the intermediate frames with interpolated or foreign frames; and a larger VLM on more windows.

5. System design

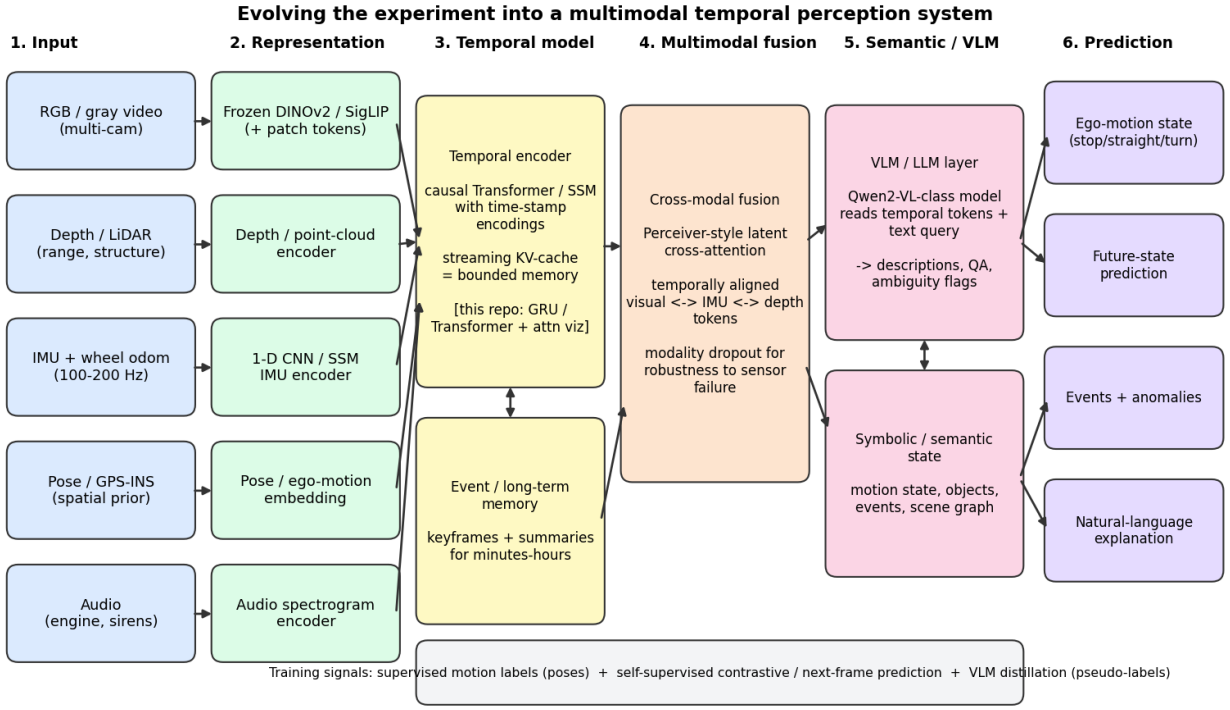


Figure 4. Proposed extension of the experiment into a multimodal perception system.

The diagram follows the order visual input, representation, temporal modelling, multimodal fusion, semantic layer, prediction. Each modality gets its own frozen or pre-trained encoder (DINOv2 or SigLIP with patch tokens for video, a depth or point-cloud encoder, a small 1-D network for IMU and wheel odometry, a pose embedding, an audio encoder) so that a sensor can be exchanged without retraining the rest. A causal temporal encoder with time-stamp encodings and a key-value cache handles irregular sampling and keeps memory bounded, with a separate long-term memory of keyframes and summaries. The results above shape two choices: the temporal encoder should receive explicit frame differences and spatially resolved features, because global embeddings mainly encode how much changed, and the IMU should anchor the motion state, since it measures yaw rate and speed directly, which makes vision-only frame comparison a redundant second source. Cross-attention fusion with modality dropout gives robustness to sensor failure, which the corruption experiment shows is needed. A VLM on top consumes temporal tokens and text queries for descriptions, question answering and ambiguity flags, while calibrated predictions come from the layers below. Training would combine pose labels, self-supervised next-frame or contrastive objectives, and pseudo-labels from the VLM.

6. Limitations

The test set is small and imbalanced (142 left and 222 right windows), the windows overlap, there are only three seeds, and the highway sequence pulls the aggregate numbers down. The label is defined by the first and last frame of the window, so a comparison of the endpoints solves the task, and the experiment cannot show deeper sequence reasoning. The stationary class was dropped for lack of data and the thresholds (3 and 9 degrees, 1.5 m) are my own choices. Frozen global DINOv2 features discard geometry. I implemented spatially pooled patch features and a larger VLM option but have not evaluated them. The frame-drop test never drops the first frame, and the VLM sample is small. The single-frame baseline depends on which frame it sees (0.418 on the last frame, 0.444 averaged), so I report both. Finally, I changed the label definition and added the delta models after looking at an earlier run, so the final test numbers are probably slightly optimistic.

7. Future improvements

Next steps are a two-frame (first and last) baseline and optical-flow inputs, the grid-pooled DINOv2 features, order-dependent targets and data (something-something, KITTI raw drives with stops), leave-one-sequence-out validation, and a 7B VLM. Of the optional directions I would try contrastive temporal pre-training, next-embedding prediction as an auxiliary loss, and IMU fusion as in Figure 4.

Third-party resources. KITTI odometry (Geiger et al.), DINOv2 (Meta AI) and Qwen2-VL (Alibaba), used frozen or zero-shot through HuggingFace transformers. All numbers in this report come from `scripts/04` to `scripts/08` in the repository.